



UNIVERZITET U NOVOM SADU FAKULTET TEHNIČKIH NAUKA



UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA
NOVI SAD

ISPITNI RAD

Kandidat: Stefan Ilić
Broj indeksa: SV12/2023

Predmet: Paralelno programiranje
Tema rada: Paralelizacija simulacije dinamike
fluida koristeći Latis-Bolcmanov metod

Mentor rada: dr Dušan Kenjić

Novi Sad, 2025.

1. Uvod

1.1 Opis problema

Problem se odnosi na simulaciju kretanja fluida u zatvorenom prostoru, pri čemu fluid može biti vazduh, gasna smeša ili tečnost u specifičnim primenama, kao što je ventilacija. Modeliranje strujanja vazduha unutar prostorija, kanala i ventilacionih sistema omogućava bolje razumevanje ponašanja fluida.

Razlozi za važnost simulacija:

- Energetska efikasnost – optimizacija ventilacionih sistema smanjuje potrošnju energije.
- Komfor i bezbednost ljudi – pravilna raspodela temperature, vlažnosti i kiseonika utiče na kvalitet boravka i zdravlje.
- Kontrola zagađenja – predviđanje širenja polutanata, alergena, dima ili aerosola, posebno važno u bolnicama i laboratorijama.
- Industrijske primene – precizna regulacija protoka vazduha u pogonima, server-sobama ili proizvodnji mikroelektronike.

Numeričke simulacije omogućavaju optimizaciju položaja otvora, brzine i pravca strujanja vazduha, smanjujući rizik od „mrtvih zona“ i omogućavajući efikasno projektovanje, kontrolu i optimizaciju ventilacionih i klimatizacionih sistema.

1.2 Teorijska podloga

Dinamika fluida može se posmatrati sa različitih nivoa preciznosti. Na atomskom nivou ($\approx 10^{-9}$ m) sistem se razmatra preko pojedinačnih čestica ili molekula, uz oslonac na Njutnove zakone. Na višem nivou ($\geq 10^{-6}$ m) koristi se mehanika kontinuuma i CFD simulacije zasnovane na Navje-Stoksovim jednačinama. Između ova dva pristupa postoji srednja skala gde nas ne zanimaju osobine pojedinačnih čestica, ali je opis kontinuuma previše grub. Tada se posmatra statistika delova fluida, kroz Bolcmanove jednačine, prateći prosečno kretanje i raspodelu brzina u „kutijama“ čestica.

Latis-Bolcmanova metoda (LBM) simulira evoluciju funkcije raspodele brzina diskretizacijom: prostor se deli na čvorove, vreme na korake Δt , a vektor brzine na ograničen skup pravaca. Svaki čvor komunicira samo sa susedima. Makroskopske veličine, poput gustine i brzine, računaju se kao momenti diskretizovane raspodele.

Osnovni koraci LBM-a su:

1. Izračunavanje momenata
2. Određivanje ekvilibrijuma
3. Lokalna kolizija (relaksacija ka ravnoteži)
4. Propagacija ka susednim čvorovima uz primenu graničnih uslova

2. Analiza problema i koncepti rešenja

Navedeni problem, zasnovan na primeni Latis-Bolcmanove metode (LBM), posebno je pogodan za paralelizaciju, jer se većina operacija odvija lokalno unutar čvorova mreže. Računanje makroskopskih veličina, određivanje ravnotežne raspodele brzina i kolizija posmatraju se na nivou pojedinačnog čvora. Čak i korak propagacije, u kome se delovi čestica prenose susednim čvorovima, lokalnog je karaktera ako se koristi dvostruki bafer – jedan iz koga se čitaju vrednosti i jedan u koji se upisuju.

Granični uslovi u LBM-u su ključni jer definišu interakciju fluida sa okolinom. Na granicama domena vrednosti se ne računaju direktno, već se zadavaju parametri koji ulaze iz spoljašnjosti. Tipično se koriste dva scenarija:

- Odbijanje čestica od zidova (bounce-back) – čestice koje bi išle ka zidu preusmeravaju se u suprotan smer, čime se izbegava klizanje uz prepreku.
- Ulazi i izlazi – za njih se zadavaju makroskopske veličine, poput brzine ili pritiska.

Rešenje paralelizacije simulacije kretanja fluida prvo je razvijeno serijski, simulirajući sve korake algoritma redom kako bi se pratilo ponašanje fluida. Zatim je implementacija prebačena na OpenCL, gde je svaki čvor mreže mapiran na jedan work item, a svaki korak algoritma izvršava se kao poseban kernel na GPU-u.

U OpenCL implementaciji prostorna mreža koja predstavlja domen fluida smeštena je u globalni bafer memorije, pri čemu svaki čvor sadrži vrednosti funkcija raspodele za sve diskretne pravce. Na ovaj način svaki work item ima pristup podacima samo za jedan čvor i izvršava sve operacije lokalno, što omogućava masivnu paralelizaciju.

Ključni koraci simulacije:

- Inicijalizacija – svakom čvoru se dodeljuju početne vrednosti gustine i brzine, iz kojih se formiraju inicijalne raspodele.
- Kolizije – primenjuje se BGK aproksimacija, koja opisuje kako funkcija raspodele teži ravnotežnom stanju.
- Propagacija (streaming) – vrednosti se transportuju u susedne čvorove duž diskretnih vektora brzine, uz upotrebu dvostrukog bafera radi izbegavanja race condition-a.
- Granični uslovi – odbijanje za zidove i zadavanje makroskopskih veličina za ulaze i izlaze, realizovani kao zasebni kernel-i.
- Izračun makroskopskih veličina – posle svake iteracije računaju se momenti funkcije raspodele, što omogućava praćenje ponašanja fluida na makroskopskom nivou.

Tehnički izazovi i optimizacije:

- Upravljanje memorijom na GPU-u – koristi se organizacija podataka u AoS (Array of Structures) ili SoA (Structure of Arrays), pri čemu SoA omogućava coalesced memory access i bolje performanse.
- Sinhronizacija – svi work item-i moraju završiti korak pre nego što počne sledeći.
- Optimizacija performansi – balansiranje broja registara, lokalne memorije i broja work group-ova za maksimalno iskorišćenje GPU-a.
- Skalabilnost – pošto je svaki čvor nezavisan, sistem se dobro skalira sa veličinom mreže, ali veći domeni zahtevaju više memorije.

3. Opis rešenja

3.1 Serijski program

Za potrebe serijskog programa napravljena je klasa LMBsys koja bi u sebi čuvala podatke neophodne za celokupnu simulaciju sistema.

```
class LMBsys {
    std::vector<uint8_t> flags;           - kontejner koji čuva učitano konfiguraciju
    std::vector<float> f;                 - Q*N, SoA layout
    std::vector<float> f_new;            - pomoćni niz za propagaciju

    std::vector<float> rho_vec;          - nizovi koji čuvaju lokalne makroskopske vrednosti
    std::vector<float> ux_vec;
    std::vector<float> uy_vec;

    float feq_at(int i, float rho, float ux, float uy) const;
                                     - funkcija za izračunavanje funkcije ekvilibrijuma distribucije,
                                     kao parametre prima poziciju, i makroskopske veličine.
    void collide(float omega);           - funkcija za računanje kolizije u svakom čvoru
    void stream();                      - funkcija za propagaciju čestica
    void applyBounceBack();             - funkcija za granični uslov za zidove (prosto odbijanje)
    void computeMacrosFrom(const std::vector<float>& src); - funkcija koja podešava polja
    void boundaryConditions(float omega, float u_in_x, float u_in_y, float rho_out);
                                     - funkcija za granične uslove ulaza/izlaza (održavanje pristika i brzine)

    void loadConfig(std::string filename); - prosto učitavanje konfiguracije iz fajla
    void refineConfig(int k);           - diskretizacija posla na zadatu preciznost
    void initialize(float u_in_x, float u_in_y); - inicijalizacija stanja na startu simulacije

    void step(float omega, float u_in_x, float u_in_y, float rho_out = 1.0f);
                                     - ceo korak koji obuhvata od propagacije do kolizije
};
```

3.2 Paralelni program

Za paralelno rešenje je odabrano korišćenje OpenCL platforme s obzirom da se priroda problema dobro uklapa na ciljeve i način rada ove platforme. Za inicijalizaciju sistema iskoristena je osnova napisana za serijski program (LBMsys). Funkcija *parallel(int STEPS, int REFINE)* je glavna funkcija koja izvodi simulaciju (prima parametre za definisanje diskretizacije i broja koraka simulacije). Inicijalizacija LBM Sistema je ista. Inicijalizacija OpenCL okruženja obuhvata selektovanje platforme i GPU uređaja, a zatim kreiranje OpenCL konteksta i *command queue*.

Naredno je kreiranje OpenCL bafera

- *f_buf, f_new_buf* – funkcije raspodele za trenutni i sledeći korak (**double buffering**).
- *rho_buf, ux_buf, uy_buf* – makroskopske promenljive: gustina i brzine.
- *flags_buf* – oznake tipova čvorova (fluid, zid, ulaz, izlaz).

Zatim, učitavanje i kompajliranje kernela. Učitavaju se i kompajliraju sledeći OpenCL kernel programi:

- *collide_kernel.cl* – kolizije (BGK aproksimacija).
- *stream_kernel.cl* – propagacija raspodela ka susednim čvorovima.
- *computeMacros_kernel.cl* – izračunavanje makroskopskih veličina.
- *bounceBack_kernel.cl* – granični uslov za zidove (bounce-back).
- *boundaryConditions_kernel.cl* – granični uslovi za ulaze/izlaze.

Ovi kernel programi su pandani istim iz serijske verzije, s tim da se ne koriste polja LBMsys objekta već kreirani baferi i globalni id unutar njih kao index.

Glavna petlja se izvodi STEPS puta i redom izvršava streaming, boundary conditions, bounce-back, compute macros, collision. Na kraju svakog koraka se vrši sinhronizacija koja je neophodna pre nego što se naredni prva akcija narednog koraka izvrši.

3.3 Vizuelizacija podataka

Vizuelizacija podataka izvršena je uz pomoć OpenCV biblioteke koja ima svoju klasu slike. Način čvorovi sistema se mogu namapirati na piksele te slike, pri čemu boju određuje tip čvora (zid – crna, ulaz – zelena, izlaz – crvena, fluid - plava), dok se za fluide primenjuje skaliranje boje od plave ka crvenoj u zavisnosti od makroskopske vrednosti brzine čvora u određenom koraku. Zbog toga što nam je, da bismo pomoću CPU-a prikazali sliku, potrebno da dobavimo sadržaj bafera za brzinu, vizuelizacija se radi na svakih 50 koraka kako ne bi značajnije ometala efikasnost simulacije.

4. Rezultati i Zaključak

4.1 Rezultati i analiza vremena izvršenja

Dimenzije	Refine	Steps	Serial	Parallel
7x5	50 (38304 čvora)	20000	18.7s	13s
16x7	50 (176904 čvora)	20000	87s	21s

4.2 Zaključak

U serijskom kodu se operacije izvode redom, pa se za svaku tačku koriste vrednosti koje su već ažurirane ili tek treba da se ažuriraju, zavisno kako je implementirano. Na GPU-u svi *work item-i* rade istovremeno, pa nema istog redosleda i koristi double buffering. LBM koristi puno sabiranja, množenja i normalizacija (računanje momenata, ekvilibrijuma, itd.), a operacije s pokretnim zarezom nisu asocijativne. Na CPU-u i GPU-u se operacije ne izvršavaju u istom redosledu, pa stoga rezultat posle više iteracija može da odluta. Razlike su male na početku, ali vremenom rastu jer se svaka iteracija oslanja na prethodnu.

Na CPU-u (serijski) redosled je 100% determinističan. Na GPU-u sve niti rade paralelno, iako double buffering uklanja *race condition-e*, sitne razlike u rasporedu instrukcija i floating-point računima čine da sistem postane nedeterminističan u smislu „bit-po-bit“ istovetnosti. Međutim, rešenje je isto u smislu globalnog ponašanja: tok strujanja, stabilizacija, raspodela brzina, formiranje vrtloga, mrtvih zona... To ne znači da se to isto rešenje može dobiti u istom broju koraka, što je kroz analizu vremena i rezultata i pokazano.

Dalji rad na ovom projektu trebalo bi da se bolje posveti obradom ulaznih podataka (otpornošću na greške, drugačijim tipom unosa) kao i povećanjem broja makroskopskih veličina koje se posmatraju i boljim mogućnostima za parametrizaciju. Uz to, veći broj i detaljnost graničnih uslova sigurno bi povećao ispravnost simulacija, a naročito bi se poboljšanje dobilo uvođenjem više dimenzija, te boljom diskretizacijom funkcije raspodele brzina.